Verilog code seven segment

```
module display(clk, q, s);
input clk;
output reg [6:0] q;
output reg [5:0] s;
reg[3:0] i;
always@(posedge clk)
begin
if(i==4'b0000)
        s=6'b000000;
case(i)
0:q=7'b0111111;
1:q=7'b0000110;
2: q=7'b1011011;
3:q=7'b1001111;
4: q=7'b1100110;
 5: q=7'b1101101;
 6: q=7'b1111101;
 7: q=7'b0000111;
 8: q=7'b1111111;
 9: q=7'b1100111;
10: q=7'b1110111;
 11: q=7'b1111100;
 12: q=7'b0111001;
 13: q=7'b1011110;
14: q=7'b1111001;
 15: q=7'b1110001;

endcase
i=i+1;
end
endmodule
```

```
#PACE: Start of PACE I/O Pin Assignments
NET "clk"  LOC = "p80"  ;
NET "q<0>"  LOC = "p114"  ;
NET "q<1>"  LOC = "p109"  ;
NET "q<2>"  LOC = "p108"  ;
NET "q<3>"  LOC = "p107"  ;
NET "q<4>"  LOC = "p102"  ;
NET "q<5>"  LOC = "p101"  ;
NET "q<6>"  LOC = "p100"  ;
NET "s<0>"  LOC = "p96"  ;
NET "s<1>"  LOC = "p95"  ;
NET "s<2>"  LOC = "p94"  ;
NET "s<3>"  LOC = "p90"  ;
NET "s<4>"  LOC = "p86"  ;
NET "s<5>"  LOC = "p81"  ;
```

Apply manual clock, to change each state press manual clockk each time.

LCD

```
Module LCD (clk, lcd_rs, lcd_rw, lcd_e, lcd_4, lcd_5, lcd_6, lcd_7);
                parameter       n = 27;
                parameter       k = 17;
 (* LOC="p80" *)  input          clk;    // synthesis attribute PERIOD clk "100.0 MHz"
                reg     [n-1:0] count=0;
                reg             lcd_busy=1;  // Lumex LCM-S01602DTR/B
                reg             lcd_stb;
                reg       [7:0] lcd_code;
                reg       [6:0] lcd_stuff;
 (* LOC="p96" *)  output reg      lcd_rs;
 (* LOC="p95" *)  output reg      lcd_rw;
 (* LOC="p97" *)  output reg      lcd_7;
 (* LOC="p100" *)  output reg      lcd_6;
 (* LOC="p101" *)  output reg      lcd_5;
 (* LOC="p102" *)  output reg      lcd_4;
 (* LOC="p94" *)  output reg      lcd_e;

  always @ (posedge clk) begin
    count <= count + 1;
    case (count[k+7:k+2])
```

```
      0: lcd_code <= 8'b00000010;    // function set
      1: lcd_code <= 8'b00000010;
      2: lcd_code <= 8'b00001100;
      3: lcd_code <= 8'b00000000;    // display on/off control
      4: lcd_code <= 8'b00001100;
      5: lcd_code <= 8'b00000000;    // display clear
      6: lcd_code <= 8'b00000001;
      7: lcd_code <= 8'b00000000;    // entry mode set
      8: lcd_code <= 8'b00000110;
     // 9: lcd_code <= 6'h24;        // H
     10: lcd_code <= 8'h21;
     //11: lcd_code <= 6'h26;        // e
     12: lcd_code <= 8'h25;
   //  13: lcd_code <= 6'h26;        // l
     14: lcd_code <= 8'h2C;
    // 15: lcd_code <= 6'h26;        // l
     16: lcd_code <= 8'h2C;
    // 17: lcd_code <= 6'h26;        // o
     18: lcd_code <= 8'h2F;
   //  19: lcd_code <= 6'h22;        //
     20: lcd_code <= 8'h20;
     //21: lcd_code <= 6'h25;        // W
     22: lcd_code <= 8'h27;
   //  23: lcd_code <= 6'h26;        // o
     24: lcd_code <= 8'h2F;
  //   25: lcd_code <= 6'h27;        // r
     26: lcd_code <= 8'h22;
//     27: lcd_code <= 6'h26;        // l
     28: lcd_code <= 8'h2C;
  //   29: lcd_code <= 6'h26;        // d
     30: lcd_code <= 8'h24;
    // 31: lcd_code <= 6'h22;        // !
     32: lcd_code <= 8'h21;
     default: lcd_code <= 8'b00010000;
   endcase
   if (lcd_rw)
     lcd_busy <= 0;
   lcd_stb <= ^count[k+1:k+0] & ~lcd_rw & lcd_busy;  // clkrate / 2^(k+2)
   lcd_stuff <= {lcd_stb,lcd_code};
   {lcd_e,lcd_rs,lcd_rw,lcd_7,lcd_6,lcd_5,lcd_4} <= lcd_stuff;
 end
```

```
endmodule
```

1. **Write HDL code to interface Hex key pad and display the key code on seven segment display.**

```
module LCD_HEX(col,row,clk,disp_sel,ss);
output [3:0] col;
initial col=4'b0001;
input [3:0] row;
input clk;
output [3:0] disp_sel;
output [6:0] ss;
reg [3:0]col;
reg[6:0] ss;
reg[3:0] disp_sel;
reg [11:0]dclk;

always @(posedge clk)
begin
dclk=dclk+1'b1;
end

always @( posedge dclk[11])
begin
col={col[2:0],col[3]};
disp_sel= 4'b1110;
```

```
end


always @*
begin
case (col)
4'b0001:case (row)
4'b0001:ss= 7'b1111110;
4'b0010:ss= 7'b0110011;
4'b0100:ss= 7'b1111111;
4'b1000:ss= 7'b1001110;
default:ss= 7'b0000000;
endcase
4'b0010: case (row)
4'b0001:ss = 7'b0110000;
4'b0010:ss = 7'b1011011;
4'b0100:ss = 7'b1111011;
4'b1000:ss = 7'b0111101;
default:ss = 7'b0000000;
endcase
4'b0100: case (row)
4'b0001:ss = 7'b1101101;
4'b0010:ss = 7'b1011111;
4'b0100:ss = 7'b1110111;
4'b1000:ss = 7'b1001111;
default:ss = 7'b0000000;
```

```
endcase

4'b1000: case (row)

4'b0001:ss = 7'b1111001;

4'b0010:ss = 7'b1110000;

4'b0100:ss = 7'b0011111;

4'b1000:ss = 7'b1000111;

default:ss = 7'b0000000;

endcase

default:ss=7'b0000000;

endcase

end

endmodule
```

procedure

1. Make the connection between FRC5 of the FPGA board to the Seven Segment.
2. Make the connection between FRC4 of the FPGA board to the Key board.
3. Make the connection between FRC6 of the FPGA board to the Dip switch.

UCF file (User constraint file)

Design Object List - I/O Pins

	I/O Name	I/O Direction	Loc	Bank	I/O Std.
	clk	Input	p52	BANK5	
	col<0>	Output	p123	BANK1	
	col<1>	Output	p131	BANK0	
	col<2>	Output	p130	BANK0	
	col<3>	Output	p137	BANK0	
	disp_sel<0>	Output	p23	BANK6	
	disp_sel<1>	Output	p24	BANK6	
	disp_sel<2>	Output	p26	BANK6	
	disp_sel<3>	Output	p27	BANK6	
	row<0>	Input	p112	BANK1	
	row<1>	Input	p116	BANK1	
	row<2>	Input	p119	BANK1	
	row<3>	Input	p118	BANK1	
	ss<0>	Output	p18	BANK7	
	ss<1>	Output	p17	BANK7	
	ss<2>	Output	p15	BANK7	
	ss<3>	Output	p14	BANK7	
	ss<4>	Output	p13	BANK7	
	ss<5>	Output	p12	BANK7	
	ss<6>	Output	p1	BANK7	

2. (A) Verilog Code to Control speed and direction of DC Motor

```
module dc_mot(clk,reset, pwm, rly, keys,dir);
input clk,reset,dir;
output [1:0] pwm;
output rly;
input [3:0] keys;
reg [1:0] pwm;
reg [16:0] div;
reg tick;
reg [7:0] counter= 0;
reg [7:0] duty_cycle;
always @ (posedge clk)
begin
div = div +1'b1;
tick= keys[3]& keys[2] & keys[1]& keys[0];
end
always @ (negedge tick)
begin
case(keys)
4'b1110: begin duty_cycle = 255;end
4'b1101: begin duty_cycle = 192;end
4'b1011: begin duty_cycle = 128;end
4'b0111: begin duty_cycle = 64;end
default: begin duty_cycle = 64;end
endcase
```

```
end
always @ (posedge div[12])
begin
if (reset==1'b1)
begin
counter = 8'b00000000;
pwm = 2'b01;
end
else
begin
counter=counter+1’b1;
if (counter <= duty_cycle)
pwm[1] = 1'b1;
else
pwm[1] = 1'b0;
end
end
assign rly = dir;
endmodule
```

procedure

1. Make the connection between FRC9 of the FPGA board to the DC motor connector.
2. Make the connection between FRC7 of the FPGA board to the Keyboard connector.
3. Make the connection between FRC1 of the FPGA board to the Dip switch connector.

UCF file(User constraint File)

```
NET "CLK" LOC = "p52" ;
NET "DIR" LOC = "p76";
NET "pwm<0>" LOC = "p4" ;
NET "pwm<1>" LOC = "p141" ;
NET "RESET" LOC = "p74" ;
NET "rly" LOC = "p44" ;
NET "Keys<0>" LOC = "p69" ;
NET "Keys<1>" LOC = "p63" ;
NET "Keys<2>" LOC = "p59" ;
NET "Keys<3>" LOC = "p57" ;
```

Or

```
module motor_clock(clk,inp,output1);
input clk,inp;
output reg [3:0] output1;
        reg [8:0] cnt=9'b000000000;
        reg [15:0] delay=16'b0000000000000000;
        reg iclk;
        reg [3:0] seq=4'b0011;
        always @(posedge clk)
                begin
                delay=delay+1;
                iclk=delay[8];
                end
        always @(posedge iclk)
        begin
        if(cnt<=9'b110010000)
        begin
//      seq={seq[0],seq[3:1]}; // rotate clockwise
        seq={seq [2:0], seq [3]}; // rotate anticlockwise
        output1=seq;
        cnt=cnt+1;
        end
        else
        begin
        output1=4'b1111;
        end
```

```
        end
endmodule


#PACE: Start of PACE I/O Pin Assignments
NET "clk"  LOC = "p80"  ;
NET "output1<0>"  LOC = "p138"  ;
NET "output1<1>"  LOC = "p132"  ;
NET "output1<2>"  LOC = "p131"  ;
NET "output1<3>"  LOC = "p125"  ;
```

(B)Write a Verilog code to control the speed and direction of the stepper motor

```
module stepper(clk, dir, d_out);
input clk, dir;
output [3:0] d_out;
reg [3:0] d_out;
reg [15:0] clk_div;
reg clk_int;
reg [3:0] shift_reg = 4'b1001;
integer i;
always@(posedge clk)
        clk_div = clk_div + 1;

always@(clk_div)
        clk_int=clk_div[15];

always@(posedge clk_int)
begin
        if (dir == 0)
                shift_reg = {shift_reg[0] , shift_reg[3:1]};
        else
                shift_reg = {shift_reg[2:0] , shift_reg[3]};
```

```
        d_out = shift_reg;
end
endmodule
```

UCF FOR STEPPER MOTOR:

```
NET "clk" LOC = "p52";
NET "dir" LOC = "p74";
NET "d_out<0>" LOC = "p141";
NET "d_out<1>"LOC = "p2";
NET "d_out<2>"LOC = "p4";
NET "d_out<3>" LOC = "p5";
```

Connections:

a. Make the connection between FRC9 of the FPGA board to the Stepper motor connector.
b. Make the connection between FRC1 of the FPGA board to the DIP switch connector.

2. Write a Verilog code to generate the following waveforms.

a. Square wave

```
module square_wave(clk, reset, dac_out);
input clk, reset;
output [7:0] dac_out;
reg [7:0] dac_out;
reg [7:0] counter = 8'b00000000;
reg [3:0]temp=3'b0;
reg enable=1'b0;
always@(posedge clk)
        temp = temp + 1;

always@(posedge temp[3])
begin
        if (reset)
                counter = 8'b00000000;
        else if ((counter < 255) && (enable == 0))
                begin
                        counter = counter+1;
```

```
                dac_out = 8'b00000000;
            end
        else if (counter == 0)
            enable = 0;
        else
            begin
                enable = 1;
                counter = counter-1;
                dac_out = 8'b11111111;
            end
end
endmodule
```

b. Triangular wave

```
module tri_wave(clk, reset, dac_out);
input clk, reset;
output [7:0] dac_out;
reg [7:0] dac_out;
reg [7:0] counter;
reg [3:0] temp = 3'b0;
reg enable = 1'b0;

always@(posedge clk)
    temp = temp + 1;

always@(posedge temp[3])
begin
    if (reset)
        counter = 8'b00000000;
    else if ((counter < 255) && (enable == 0))
        begin
            counter = counter+1;
            dac_out = counter;
        end
    else if (counter == 0)
        enable = 0;
    else
        begin
            enable = 1;
            counter = counter-1;
            dac_out = counter;
        end
end
```

```
endmodule
```

c. Sawtooth wave

```
module saw_wave(clk, reset, dac_out);
input clk, reset;
output [7:0] dac_out;
reg [7:0] dac_out;
reg [7:0] counter;
reg [3:0]temp = 3'b0;
always@(posedge clk)
        temp = temp + 1;

always@(posedge temp[3])
begin
        if (reset)
                counter = 8'b00000000;
        else
                counter = counter+1;
        dac_out = counter;
end
endmodule
```

d. Sine wave

```
module sine_wave(clk, reset, dac_out);
input clk, reset;
output [7:0] dac_out;
reg [7:0] dac_out;
reg [7:0] sine [0:71];
reg [3:0] temp = 3'b0;
integer i;

always@(posedge clk)
        temp = temp + 1;

always@( posedge temp[3])
begin
        if (reset)
        begin
                sine[0]= 128;           // 128 + 128 sinθ
                sine[1] = 139;
                sine[2] = 150;
                sine[3] = 161;
```

```
sine[4] = 171;
sine[5] = 182;
sine[6] = 192;
sine[7] = 201;
sine[8] = 210;
sine[9] = 218;
sine[10] = 226;
sine[11] = 232;
sine[12] = 238;
sine[13] = 244;
sine[14] = 248;
sine[15] = 251;
sine[16] = 254;
sine[17] = 255;
sine[18] = 255;
sine[19] = 255;
sine[20] = 254;
sine[21] = 251;
sine[22] = 248;
sine[23] = 244;
sine[24] = 238;
sine[25] = 232;
sine[26] = 226;
sine[27] = 218;
sine[28] = 210;
sine[29] = 201;
sine[30] = 192;
sine[31] = 182;
sine[32] = 171;
sine[33] = 161;
sine[34] = 150;
sine[35] = 139;
sine[36] = 128;
sine[37] = 116;
sine[38] = 105;
sine[39] = 94;
sine[40] = 84;
sine[41] = 73;
sine[42] = 64;
sine[43] = 55;
sine[44] = 45;
sine[45] = 37;
sine[46] = 29;
sine[47] = 23;
```

```
                sine[48] = 17;
                sine[49] = 11;
                sine[50] = 7;
                sine[51] = 4;
                sine[52] = 1;
                sine[53] = 0;
                sine[54] = 0;
                sine[55] = 0;
                sine[56] = 1;
                sine[57] = 4;
                sine[58] = 7;
                sine[59] = 11;
                sine[60] = 17;
                sine[61] = 23;
                sine[62] = 29;
                sine[63] = 37;
                sine[64] = 45;
                sine[65] = 55;
                sine[66] = 64;
                sine[67] = 73;
                sine[68] = 84;
                sine[69] = 94;
                sine[70] = 105;
                sine[71] = 116;
        end
        else
        begin
                dac_out = sine[i];
                 i = i+ 1;
                 if(i == 72)
                        i = 0;
        end
end
endmodule
```

UCF FOR DAC:

```
NET "clk" LOC= "p52";
NET "dac_out<0>" LOC= "p1";
NET "dac_out<1>" LOC= "p12";
NET "dac_out<2>" LOC= "p13";
NET "dac_out<3>" LOC= "p14";
NET "dac_out<4>" LOC= "p15";
```

```
NET "dac_out<5>" LOC= "p17";
NET "dac_out<6>" LOC= "p18";
NET "dac_out<7>" LOC= "p21";
NET "reset" LOC= "p74";
```

Connections:

a) Make the connection between FRC5 of the FPGA board to the DAC connector.
b) Make the connection between FRC1 of the FPGA board to the DIP switch connector.